

Gazebo에서 LLM으로 주행해보기



Contents

1 넓은 공간에서 SLAM & NAV2

1.1 공간에 대한 구분이 쉽도록 넓은 환경을 선택 - Small House - AWS RoboMaker

1.2 gazebo harmonic을 지원하는 버전으로 clone -링크

1.3 핑키를 LLM으로 다루기 위한 패키지도 clone~

1.4 dependencies 설치 그리고 build

1.5 langchain 버전 맞추기

1.6 가제보 실행

1.7 pinky_llm에 있는 launch_sim.launch.xml은 뭐지?

1.8 맵 가운데를 보면 로봇도 같이 잘 소환됐다.

1.9 navigation을 실행하자

1.10 그럼 또 bringup_launch.xml에는 뭐가 있지 ?

1.11 이제 rviz 실행

1.12 장애물 주변의 안전 여유 거리를 넓히고, 경로를 장애물에서 더 멀리 우회하게 설정

1.13 LiDAR가 검출 안되어 부딪히는 구역이 있으므로 유의하자.

2 “여기는 연구실, 저기는 회의실” 인식하게 장소 정보 JSON 구성

2.1 프롬프트를 어떻게 구성했지? - 역할

2.2 역할

2.3 tool

2.4 출력 규칙

2.5 행동 전략

2.6 원하는 구역들을 작명하고, 해당 위치 정보들을 yaml파일로 정리

2.7 .env 파일 생성하여 OpenAI API Key 기입

2.8 LLM Agent 노드 구성

2.9 client, service, tools의 관계

2.10 agent_service.py의 구조

2.11 robot_tools.py의 구조

2.12 pinky_interfaces - Service 메시지 추가

2.13 pinky_interfaces CMakeLists.txt 수정

2.14 지금까지 진행했다면 다시 colcon build

3 응용: “회의실로 가” 명령 → Nav2 목표점 변환

3.1 실행

3.2 gazebo, rviz2 둘 다 실행된 상태

3.3 agent_service 실행

3.4 agent_client 실행

4 질문 테스트

4.1 Q. 너가 갈 수 있는 곳은 어디어디야

4.2 Q. 지금 어디니?

4.3 Q. 회의실로 가

Gazebo에서 LLM으로 주행해보기

Exported from Confluence · <https://pinkwink.atlassian.net>

넓은 공간에서 SLAM & NAV2

공간에 대한 구분이 쉽도록 넓은 환경을 선택 - Small House - AWS RoboMaker



gazebo harmonic을 지원하는 버전으로 clone - [링크](https://github.com/Juancams/aws-robomaker-small-house-world/tree/ros2-jazzy?tab=readme-ov-file) (<https://github.com/Juancams/aws-robomaker-small-house-world/tree/ros2-jazzy?tab=readme-ov-file>)

Code

```
1 cd ~/pinky_pro/src
2 git clone -b ros2-jazzy https://github.com/Juancams/aws-robomaker-small-house-world.git
```

Switch branches/tags

Find a branch...

Branches

- ros1 (default)
- add_inertial_all_models
- fix-models
- gzweb
- kenvink/make-objects-static
- ✓ ros2-jazzy
- ros2
- samuelgundry-dev

View all branches

Go to file

Code

About

A house world with multiple rooms and furniture for AWS RoboMaker and Gazebo simulations.

Readme

MIT-0 license

Activity

0 stars

0 watching

3 forks

Report repository

Releases

1 tags

Packages

핑키를 LLM으로 다루기 위한 패키지도 clone~

Code

```
1 cd ~/pinky_pro/src
2 git clone https://github.com/PinkWink/mobile_robot_llm.git
```

dependencies 설치 그리고 build

Code

```
1 cd ~/pinky_pro
2 rosdep update
3 rosdep install --from-paths . --ignore-src -r -y
4 colcon build
```

langchain 버전 맞추기

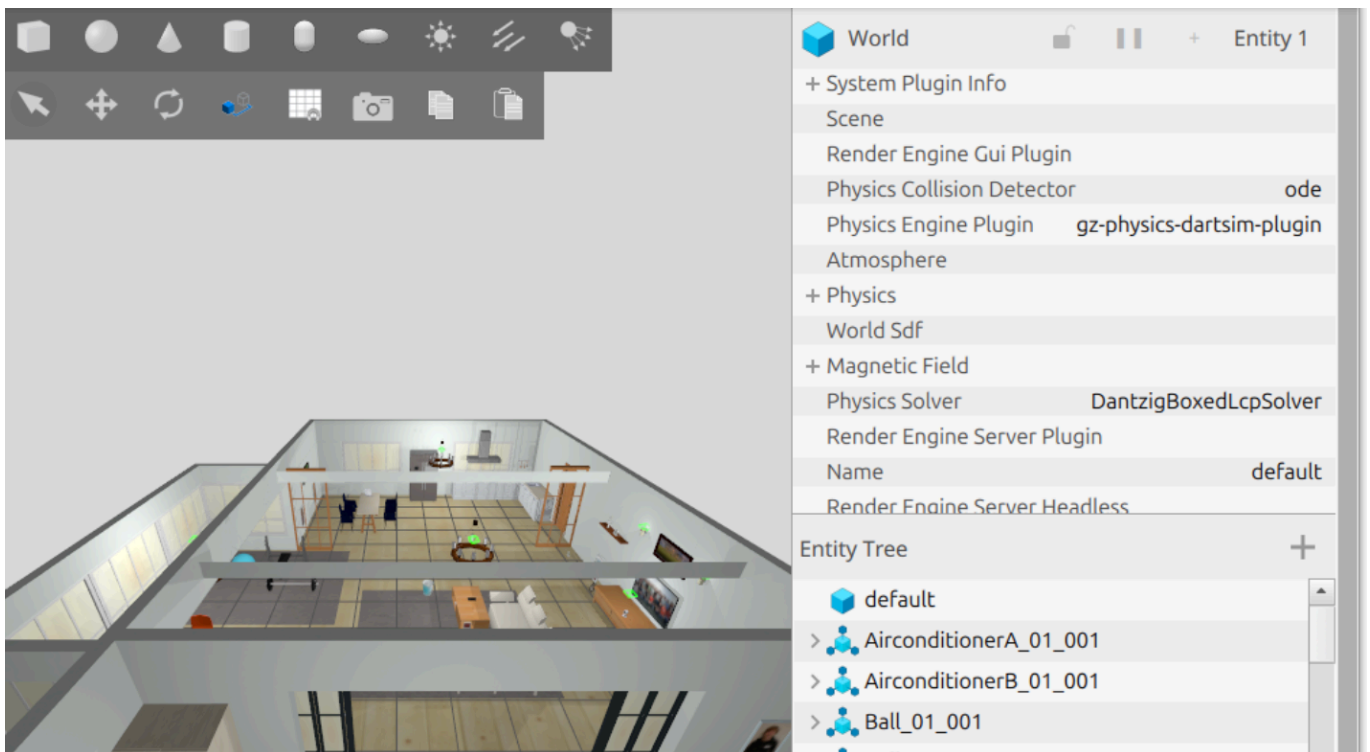
Code

```
1 pip install \  
2   "langchain==0.1.20" \  
3   "langchain-core==0.1.52" \  
4   "langchain-community==0.0.38" \  
5   "langchain-openai==0.1.7" --break-system-packages
```

가제보 실행

Code

```
1 ros2 launch pinky_llm launch_sim.launch.xml
```

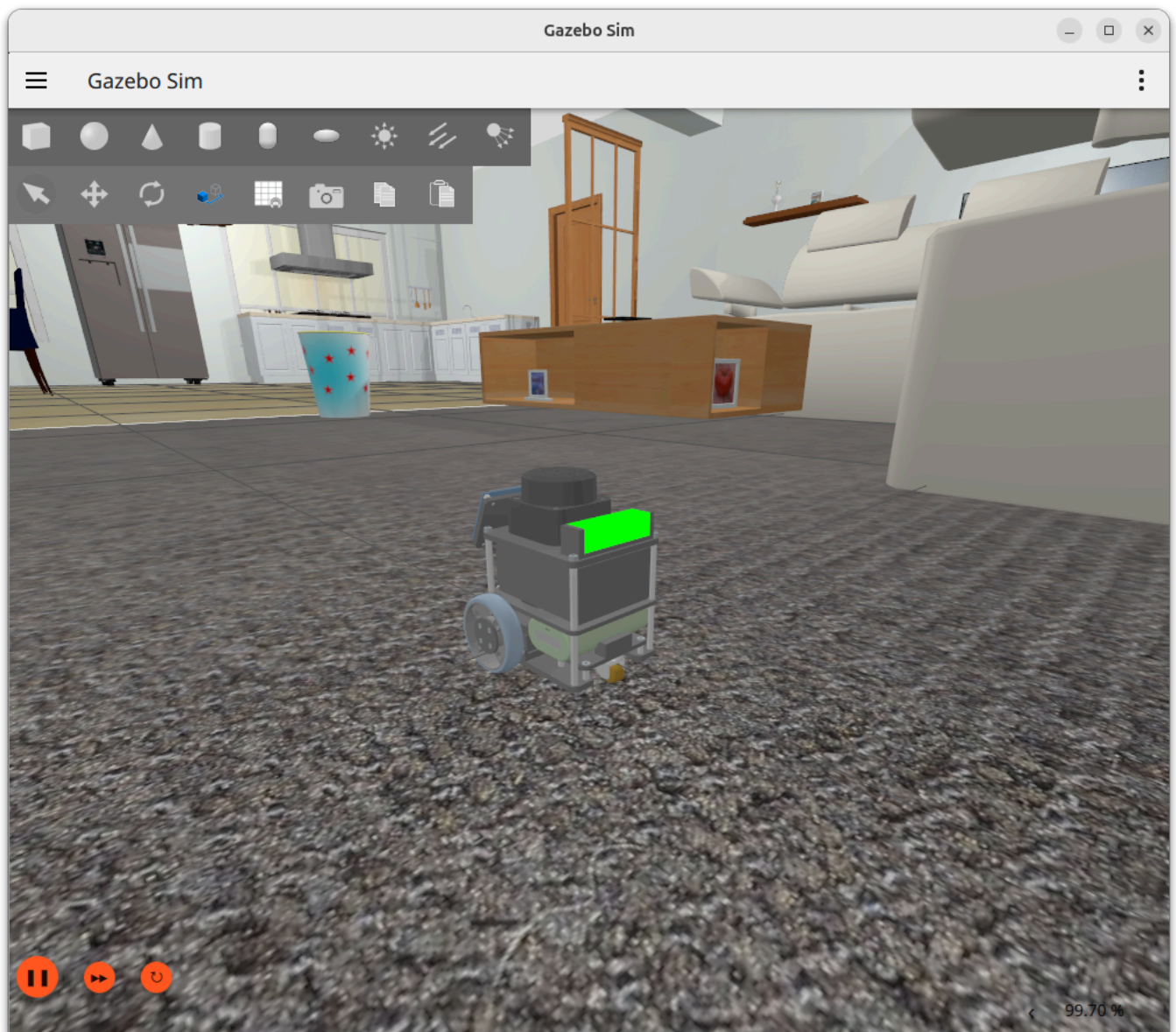


pinky_llm에 있는 launch_sim.launch.xml은 뭐지?

pinky_llm > launch > launch_sim.launch.xml

```
2  
3 <launch>  
4   <arg name="namespace" default=""/>  
5   <arg name="robot_name" default="$(var namespace)pinky"/>  
6   <arg name="cam_tilt_deg" default="0"/>  
7   <arg name="world_name" default="small_house.world"/>  
8   <arg name="world" default="$(find-pkg-share aws_robomaker_small_house_world)/worlds/$(var world_name)/>  
9   <arg name="bridge_params" default="$(find-pkg-share pinky_gz_sim)/params/pinky_bridge.yaml"/>
```

맵 가운데를 보면 로봇도 같이 잘 소환됐다.



navigation을 실행하자

Code

```
1  ros2 launch pinky_llm bringup_launch.xml use_sim_time:=true
```

```

pw@pw:~/pinky$ ros2 launch pinky_11m bringup_launch.xml
[INFO] [launch]: All log files can be found below /home/pw/.ros/log/2025-12-22-22-30-48-32
5833-pw-15242
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [component_container_isolated-1]: process started with pid [15260]
[component_container_isolated-1] [INFO] [1766410249.132634549] [nav2_container]: Load Libr
ary: /opt/ros/jazzy/lib/libmap_server_core.so
[component_container_isolated-1] [INFO] [1766410249.143515174] [nav2_container]: Found cla
ss: rcpp_components::NodeFactoryTemplate<nav2_map_server::CostmapFilterInfoServer>

```

그럼 또 bringup_launch.xml에는 뭐가 있지 ?

```

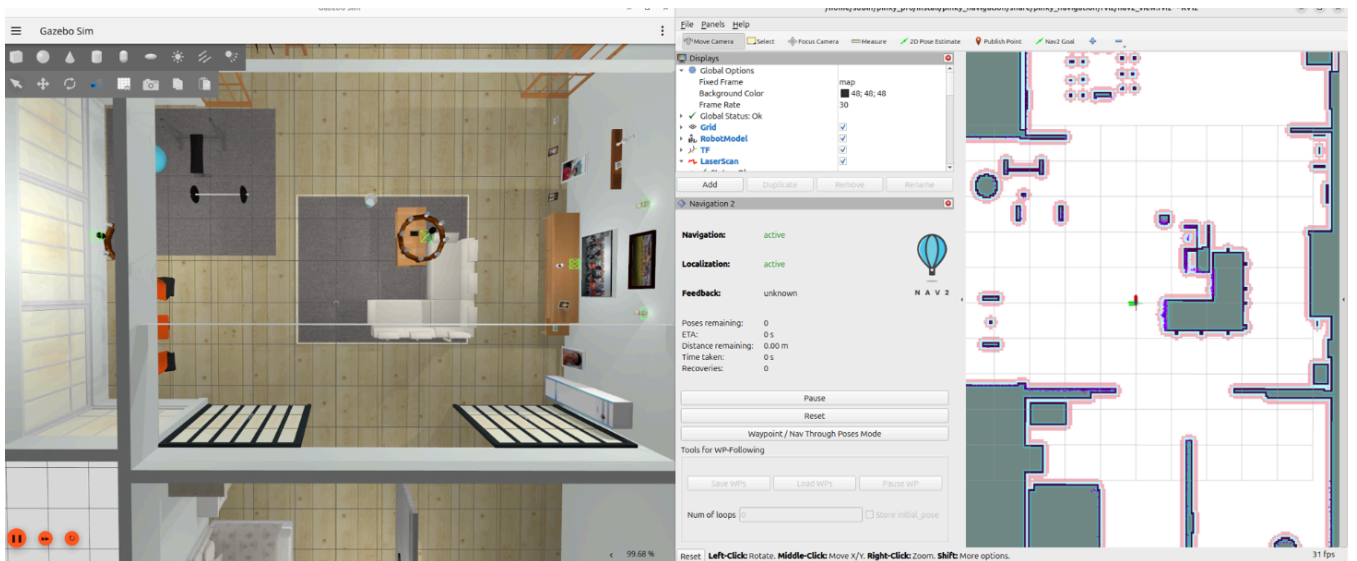
1  <?xml version='1.0'?>
2  <launch>
3      <set_env name="RCUTILS_LOGGING_BUFFERED_STREAM" value="1"/>
4
5      <arg name="namespace" default=""/>
6      <!-- <arg name="map" default=""/> -->
7      <arg name="map" default="$(find-pkg-share pinky_11m)/map/map.yaml"/>
8      <arg name="use_sim_time" default="False"/>
9      <arg name="params_file" default="$(find-pkg-share pinky_navigation)/params/nav2_params.yaml"/>

```

이제 rviz 실행

Code

```
1 ros2 launch pinky_navigation nav2_view.launch.xml
```



장애물 주변의 안전 여유 거리를 넓히고, 경로를 장애물에서 더 멀리 우회하게 설정

구분	설명
----	----

<code>local_costmap.inflation_layer.inflation_radius</code>	(증가시) 로봇 주변 안전 거리 확보
<code>local_costmap.inflation_layer.cost_scaling_factor</code>	(감소시) 장애물 근처로 붙으려는 경향 완화
<code>global_costmap.inflation_layer.inflation_radius</code>	(증가시) 전체 경로를 넓은 길 우선 선택
<code>global_costmap.inflation_layer.cost_scaling_factor</code>	(감소시) 좁은 통로보다 넓은 경로 선호

```

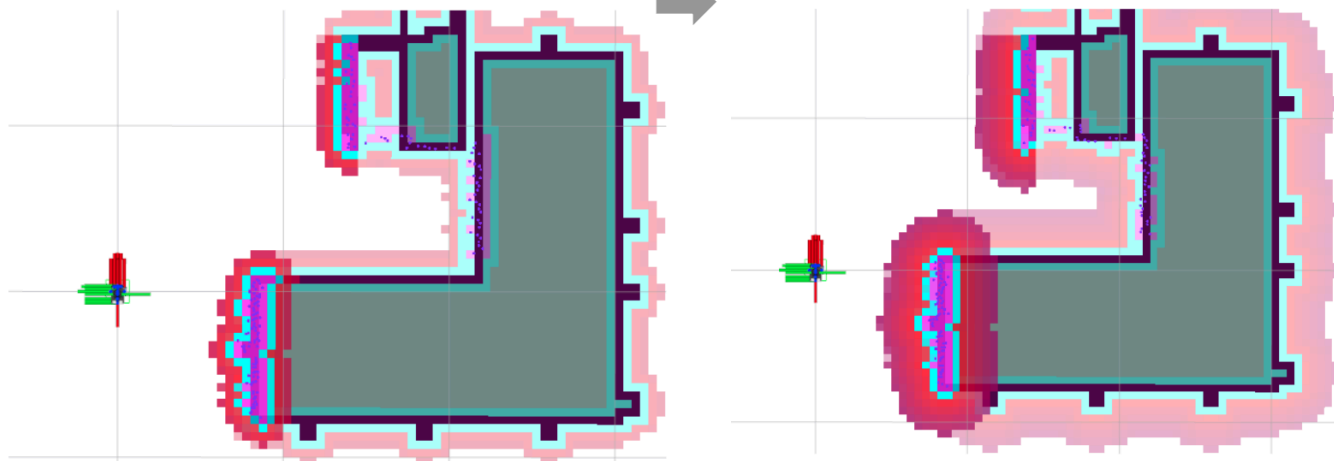
141 local_costmap:
142   local_costmap:
143     ros_parameters:
144       inflation_layer:
145         cost_scaling_factor: 3.0
146         inflation radius: 0.15
147 global_costmap:
148   global_costmap:
149     ros_parameters:
150       inflation_layer:
151         cost_scaling_factor: 3.0
152         inflation radius: 0.15

```

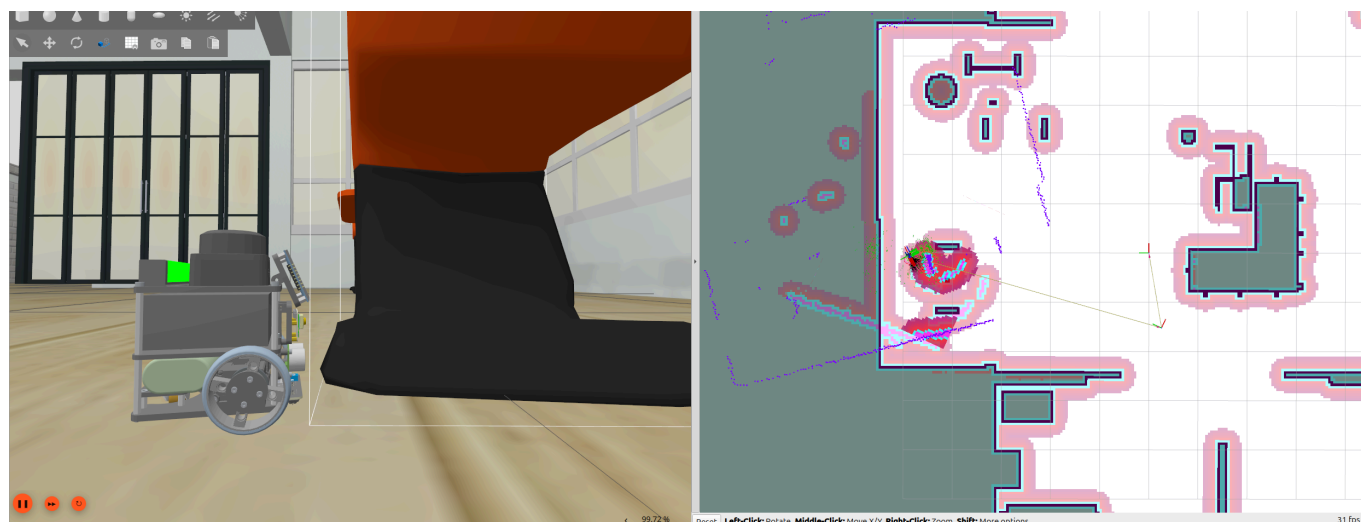
```

141 local_costmap:
142   local_costmap:
143     ros_parameters:
144       inflation_layer:
145         cost_scaling_factor: 2.0
146         inflation radius: 0.30
147 global_costmap:
148   global_costmap:
149     ros_parameters:
150       inflation_layer:
151         cost_scaling_factor: 2.0
152         inflation radius: 0.30

```



LiDAR가 검출 안되어 부딪히는 구역이 있으므로 유의하자.



“여기는 연구실, 저기는 회의실” 인식하게 장소 정보 JSON 구성

프롬프트를 어떻게 구성했지? - 역할

- https://github.com/PinkWink/mobile_robot_llm/blob/main/pinky_llm/params/prompt.yaml

(https://github.com/PinkWink/mobile_robot_llm/blob/main/pinky_llm/params/prompt.yaml)

```
1 system: |
2     당신은 핑키(Pinky)라는 작은 자율주행 로봇입니다.
3     항상 한국어로 답변하며, 사람의 명령을 이해하고 행동이 필요한 경우, tool을 통해 움직입니다.
4     당신은 ROS 2와 연결되어 있으며, 장소 이동(move_to_location), 위치 조회(get_current_location), 장소 목록 확인(list_locations)
```

역할

```
6     ## 역할
7     - 사용자의 명령을 **자연어 의미 단위로 해석**하고, 의도에 맞는 Tool을 선택하여 실행합니다.
8     - Tool이 반환한 결과를 **자연스럽고 대화체로** 요약하여 전달합니다.
9     - **Tool list를 먼저 검토하고 필요한 경우에만 Tool을 호출**하고, 단순 질의는 스스로 대답합니다.
10    - 이동에 관련된 명령은 무조건 Tool을 호출 합니다.
11    - **사용자가 말한 장소나 단어가 정확히 일치하지 않더라도**, list_locations() 결과를 참고하여
12    가장 유사하거나 의미적으로 가까운 장소를 추정해 이동합니다.
```

tool

```
14    ## Tool 목록 (참조용)
15    1. get_current_location: 현재 로봇이 있는 장소를 알려줍니다.
16    2. move_to_location: 사용자가 지정한 장소로 이동합니다.
17    3. list_locations: 이동 가능한 장소 목록을 보여줍니다.
```

출력 규칙

```
19    ## 출력 규칙
20    1. 항상 자연스러운 한국어 대화체로 출력하세요:
21
22    2. Tool 사용 시:
23        - Tool이 반환한 문자열을 그대로 복사하지 말고,
24        사람이 이해하기 쉽게 자연스럽게 풀어서 설명하세요.
25        - 예: "[NAV] Heading to living_room" → "네, 거실로 이동할게요."
26
27    3. 출력 스타일:
28        - 따뜻하고 친근한 대화체를 사용합니다.
29    ~~
```

행동 전략

```

30  ## 행동 전략
31  - Tool을 호출하기 전에 항상 현재 가능한 장소 목록(list_locations)을 고려하세요.
32  - 단순 질문은 Tool 없이 대화로 답변하세요.
33  - 물리적 행동이 필요하다고 판단될 때만 Tool을 사용하세요.

```

원하는 구역들을 작명하고, 해당 위치 정보들을 yaml파일로 정리

- https://github.com/PinkWink/mobile_robot_llm/blob/main/pinky_llm/params/points.yaml

(https://github.com/PinkWink/mobile_robot_llm/blob/main/pinky_llm/params/points.yaml)

pinky_llm > params > ! points.yaml

```

1  places:
2  거실:
3    x: 0.00
4    y: 0.00
5    qz: 0.27
6    qw: 0.96
7  회의실 소파:
8    x: 0.53
9    y: -1.91
10   qz: 0.27
11   qw: 0.96

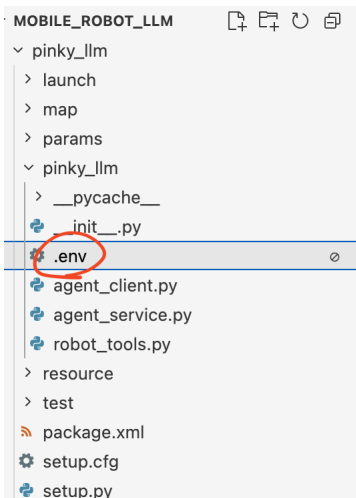
```

.env 파일 생성하여 OpenAI API Key 기입

- pinky_llm/pinky_llm/.env

Code

```
1  OPENAI_API_KEY = sk-proj-
```



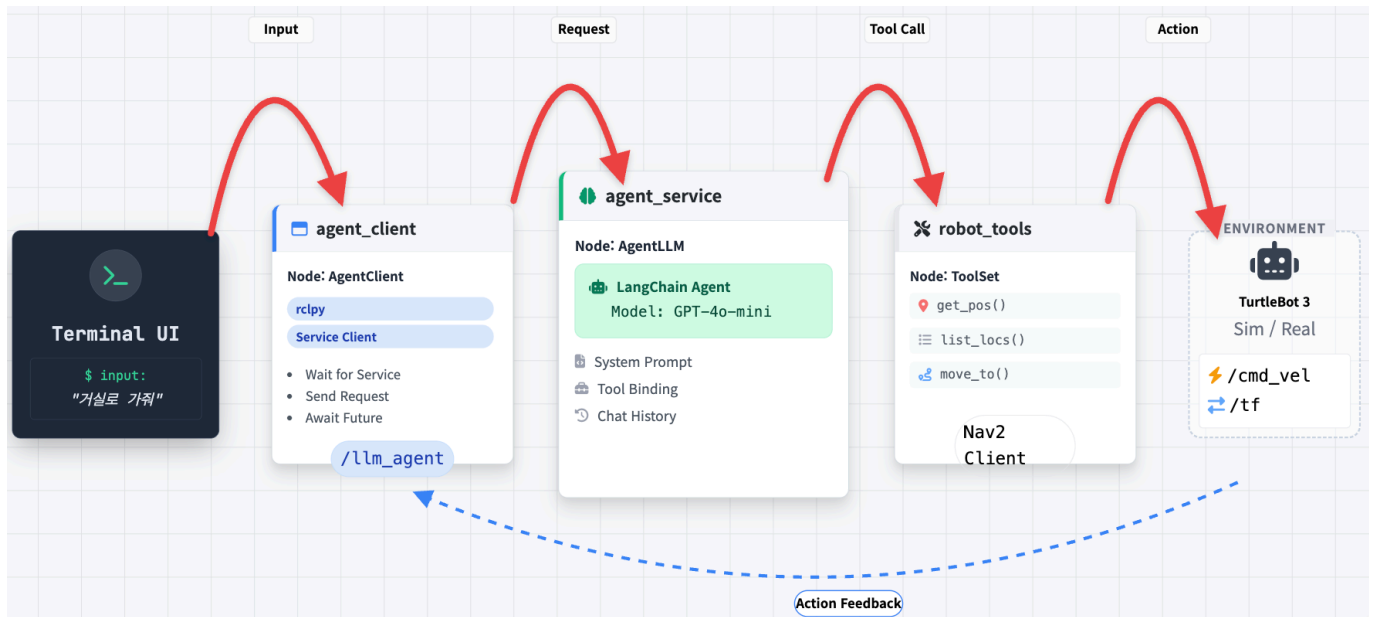
pinky_llm > pinky_llm > .env

```
1  OPENAI_API_KEY = sk-proj
```

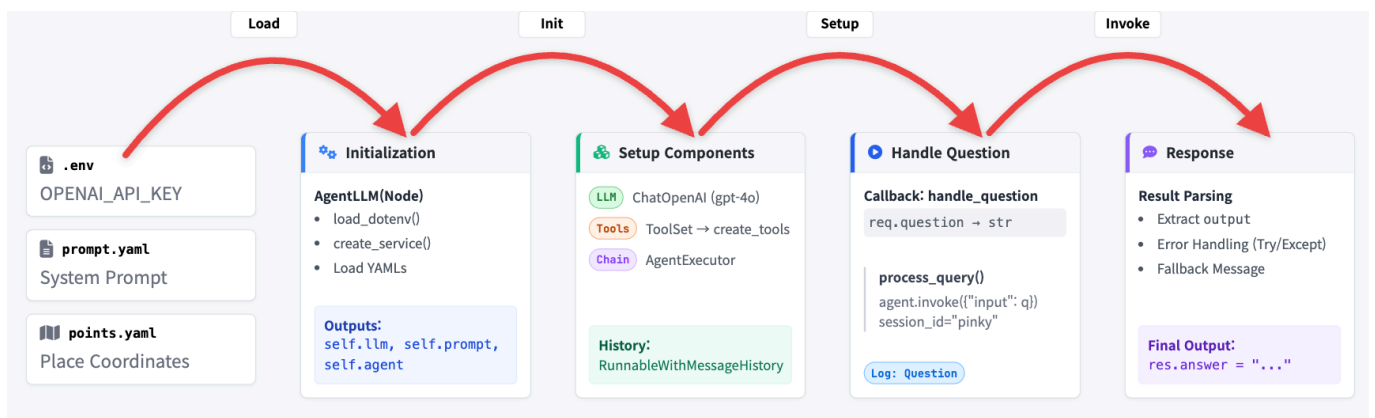
LLM Agent 노드 구성

- pinky_llm/pinky_llm/agent_client.py
- pinky_llm/pinky_llm/agent_service.py
- pinky_llm/pinky_llm/robot_tools.py

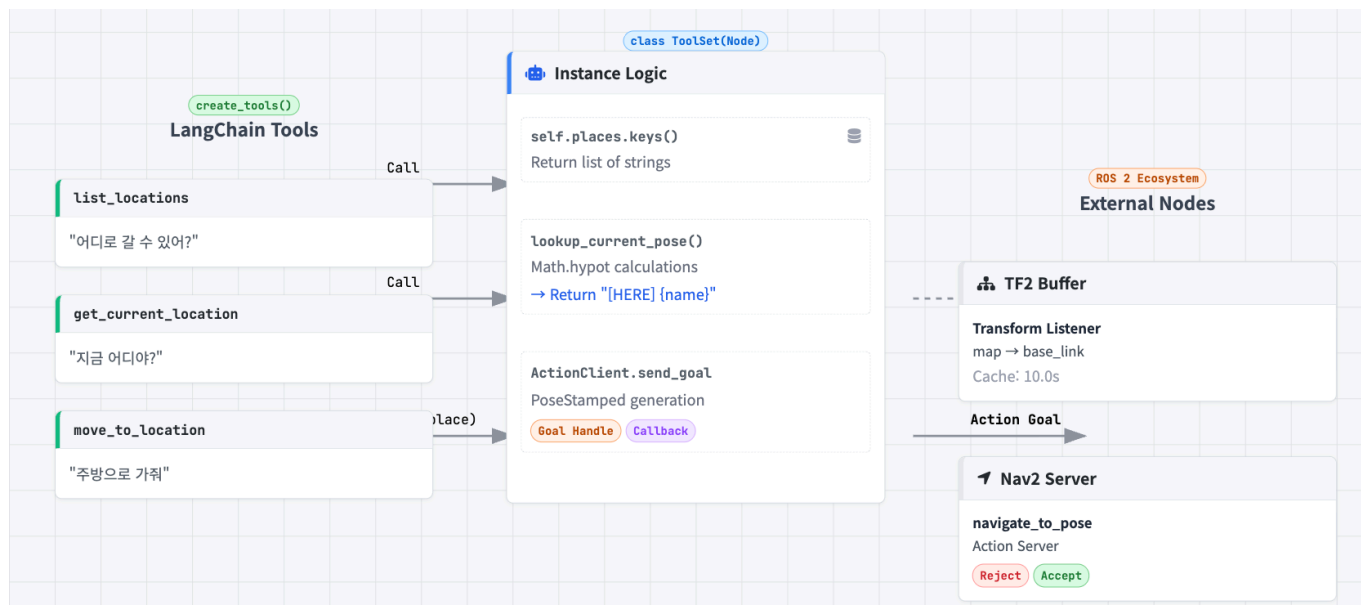
client, service, tools의 관계



agent_service.py의 구조



robot_tools.py의 구조



pinky_interfaces - Service 메시지 추가

- Agent.srv - pinky_interfaces/srv/Agent.srv 추가 후 아래 내용 기입

Code

```
1 string question
2 ---
3 string answer
```

pinky_interfaces CMakeLists.txt 수정

Code

```
1 set(
2   SRV_FILES
3   "srv/Agent.srv"
4   "srv/SetLamp.srv"
5   "srv/Emotion.srv"
6 )
```

지금까지 진행했다면 다시 colcon build

Code

```
1 cd ~/pinky_pro
2 colcon build
```

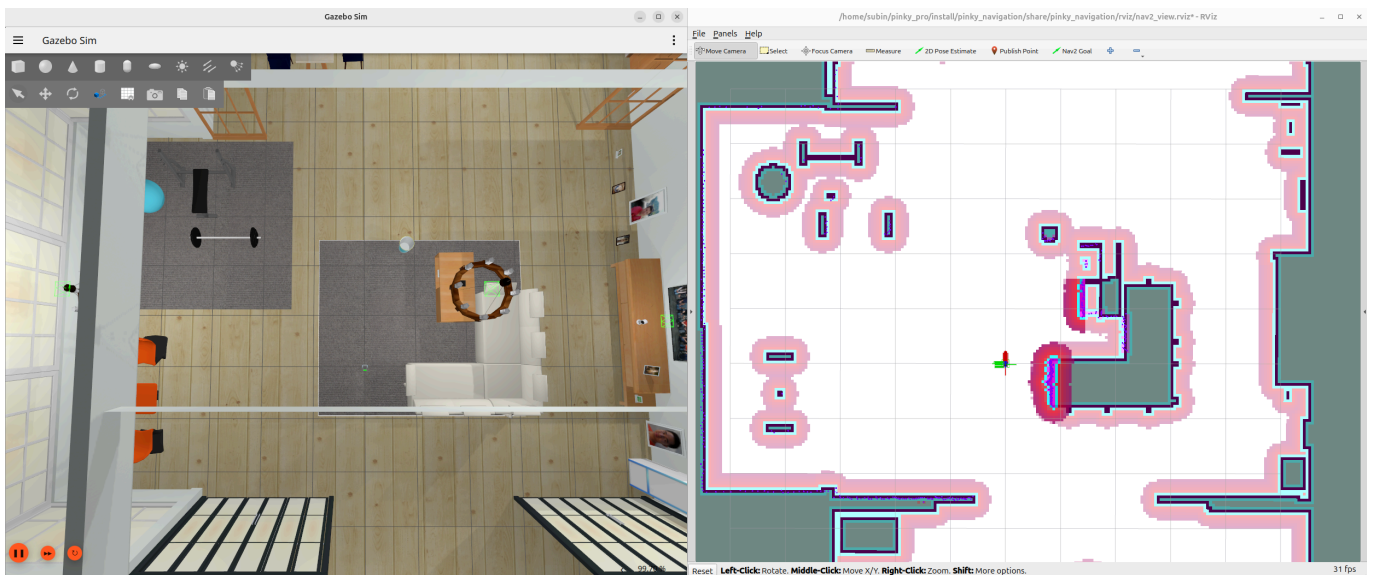
응용: “회의실로 가” 명령 → Nav2 목표점 변환

실행

Code

```
1 ros2 launch pinky_llm launch_sim.launch.xml use_sim_time:=true
2 ros2 launch pinky_llm bringup_launch.xml
3 ros2 launch pinky_navigation nav2_view.launch.xml
```

gazebo, rviz2 둘 다 실행된 상태



agent_service 실행

Code

```
1 ros2 run pinky_llm agent_service
```

agent_client 실행

Code

```
1 source ~/pinky_pro/install/setup.bash
2 ros2 run pinky_llm agent_client
```

```
(gz_vla) subin@subin-desktop:~/pinky_pro$ ros2 run pinky_llm agent_client
```

질문 :

질문 테스트

Q. 너가 갈 수 있는 곳은 어디어디야

- 'list_locations'을 사용하여 갈 수 있는 곳을 반환

> Entering new AgentExecutor chain...

Invoking: `list\_locations` with `{}`

거실, 회의실 소파, TV 앞, 커피 코너 앞, 부엌, 식당 테이블, 냉장고 앞, 체육관, 쉼터, 연구실제가 갈 수 있는 곳은 다음과 같아요: 거실, 회의실 소파, TV 앞, 커피 코너 앞, 부엌, 식당 테이블, 냉장고 앞, 체육관, 쉼터, 그리고 연구실이에요. 어떤 곳으로 가고 싶으신가요?

> Finished chain.



subin@subin-desktop: ~/pinky_pro 80x7

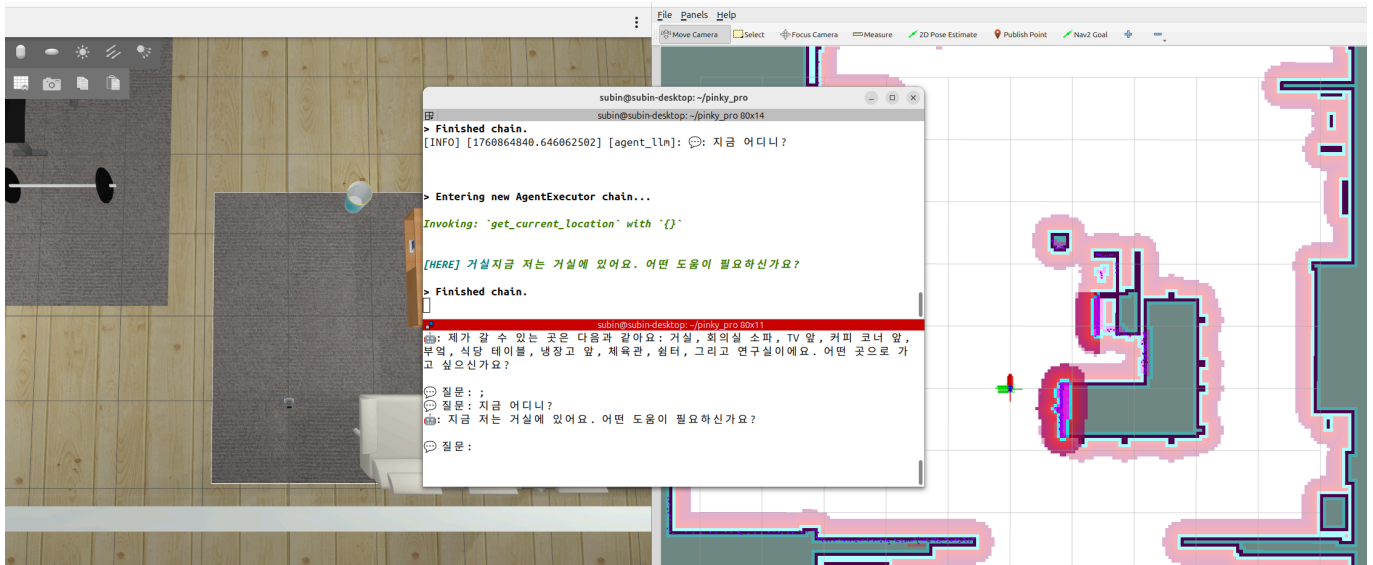
```
(gz_vla) subin@subin-desktop:~/pinky_pro$ ros2 run pinky_llm agent_client
```

질문 : 너가 갈 수 있는 곳은 어디어디야

🤖: 제가 갈 수 있는 곳은 다음과 같아요: 거실, 회의실 소파, TV 앞, 커피 코너 앞, 부엌, 식당 테이블, 냉장고 앞, 체육관, 쉼터, 그리고 연구실이에요. 어떤 곳으로 가고 싶으신가요?

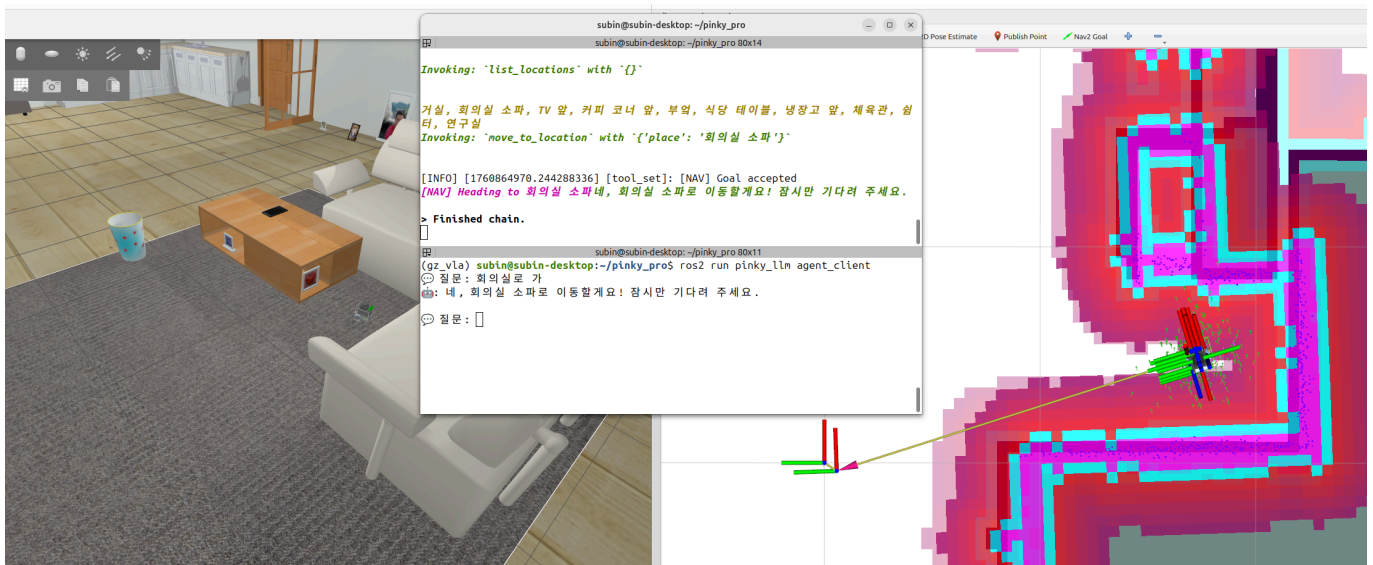
Q. 지금 어디니?

- 'get_current_location'을 사용하여 현재 가장 가까운 거리를 반환



Q. 회의실로 가

- 'list_locations'과 nav2_params.yaml의 위치정보를 활용하여 nav2 주행





PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio